Chair of Computer Architecture and Operating Systems
TUM Campus Heilbronn – TUM School of Computation, Information and Technology (CIT)
Technical University of Munich

Design and Verification of a Pipelined RISC-V Processor on PYNQ-Z2 FPGA

Saumil Savani ✉ **and Zhenkai Lin** ✉

✉ go69jal@mytum.de
✉ animenl.lin@tum.de

6 August 2026

Abstract — This report documents a pipelined RISC-V softcore implemented on the PYNQ-Z2 FPGA and verified through a hardware-in-the-loop workflow. The Zynq processing system (PS) loads the overlay, injects a machine-code program and test data into BRAM, controls the processor reset through AXI GPIO, and validates the output after execution. The programmable logic (PL) sorts 32 signed 32-bit integers in place and signals completion through a dedicated data-memory word. Reverse-order and signed-integer board tests pass. The routed Vivado implementation meets the 50 MHz constraint with +7.081 ns worst setup slack.

1 Design Overview

1.1 Objective and functionality

The project goal was to take a pipelined RISC-V processor beyond RTL-only verification and make it a repeatable system on the PYNQ-Z2. The completed design combines a custom processor in PL with a Python/PYNQ test harness running on the ARM-based PS. The PS loads the bitstream, initializes instruction and data memories, releases reset, waits for the program to terminate, and compares the final memory contents with a Python reference result.

The benchmark is an in-place sort of exactly 32 signed 32-bit integers. It exercises arithmetic, loads, stores, branches, comparisons, jumps, data hazards, and control hazards. Completion is reported by a magic value written to a reserved data-memory location. This makes a complete run observable without relying on a debugger or a manually timed delay.

Before execution, the PYNQ notebook verifies that the hardware handoff, bitstream, and machine-code image are present in the notebook directory (Figure 1). It then loads the overlay and checks that the overlay reports a successful load. This check separates deployment errors, such as a missing bitstream or stale hardware description, from processor or software errors later in the workflow.

1.2 System structure

Figure 2 shows the PS–PL partition. The PS accesses two independent BRAM regions through AXI BRAM controllers and drives a reset signal through AXI GPIO. The RISC-V wrapper fetches program words from instruction BRAM and accesses the data BRAM for loads and stores. The separate instruction and data memories used by the board design avoid an instruction-fetch/data-access structural conflict at the external memory interface.

1.3 Role of the JTAG-to-AXI Master (`jtag_axi_0`):

As visible in the IP block design, a `jtag_axi_0` (JTAG to AXI Master) IP block is integrated into the AXI interconnect. Under standard PYNQ operations, memory initialization and execution control are handled entirely by the ARM (PS) via AXI. However, the JTAG-to-AXI Master was intentionally included during hardware development as an independent direct debugging interface. It allows developers using Vivado Hardware Manager to issue raw AXI read and write transactions directly to the instruction BRAM, data BRAM, and GPIO reset registers via the physical JTAG port. This provides a hardware back-door to inspect system state and verify memory integrity independently of the ARM PS driver stack.

1.4 Memory map and execution protocol

The notebook uses the address map in Table 1. It first holds the CPU in reset, writes the instruction image and input array, clears the DONE location at data offset `0x1000`, and performs readback checks (Figure 3, Figure 4). It then releases reset. The program writes `0xCAFEBABE` at the reserved location when sorting is complete; the host polls this word with a timeout and finally checks the 32 output words.

The overlay metadata is checked against these expected base addresses before creating the PYNQ `MMIO` objects (Figure 3). The CPU is then held in reset

Figure 1 PYNQ deployment preflight. The notebook confirms the presence of `design.bit`, `design.hwh`, and `test_sort.hex`, then loads the FPGA overlay successfully.

Table 1 Runtime memory map used by the PYNQ verification notebook.

Region	Base address
Instruction BRAM	`0x40000000`
Reset GPIO	`0x41200000`
Data BRAM	`0x42000000`
DONE word in data BRAM	offset `0x1000`

while the BRAM contents are initialized. This makes the address-map verification an explicit part of the hardware/software interface rather than an assumed configuration detail.

2 Module Description and Implementation

2.1 Five-stage pipeline

The processor follows the conventional IF–ID–EX–MEM–WB organization. Pipeline registers separate the stages so that several instructions can be in flight simultaneously. The implementation supports the RV32I subset needed by the test program; its encoding and architectural conventions follow the RISC-V unprivileged specification [1].

Instruction fetch (IF). The program counter resets to zero. Under sequential execution, the next address is PC+4. A taken branch or jump replaces this value with the execute-stage target address. When a control transfer resolves, younger wrong-path state is flushed.

Decode and register read (ID). The decoder extracts opcode, source and destination registers, function fields, and the appropriate sign-extended immediate. The register file provides two combinational read ports and ignores writes to x0. The supplied RTL writes the register file on the falling clock edge, allowing a subsequently decoded instruction to observe a recently written value.

Execute (EX). The ALU performs arithmetic, logic, shifts, and signed or unsigned comparisons. A multiplexer selects either a forwarded register operand or an immediate as the second ALU input. Branch conditions and branch target addresses are also formed here.

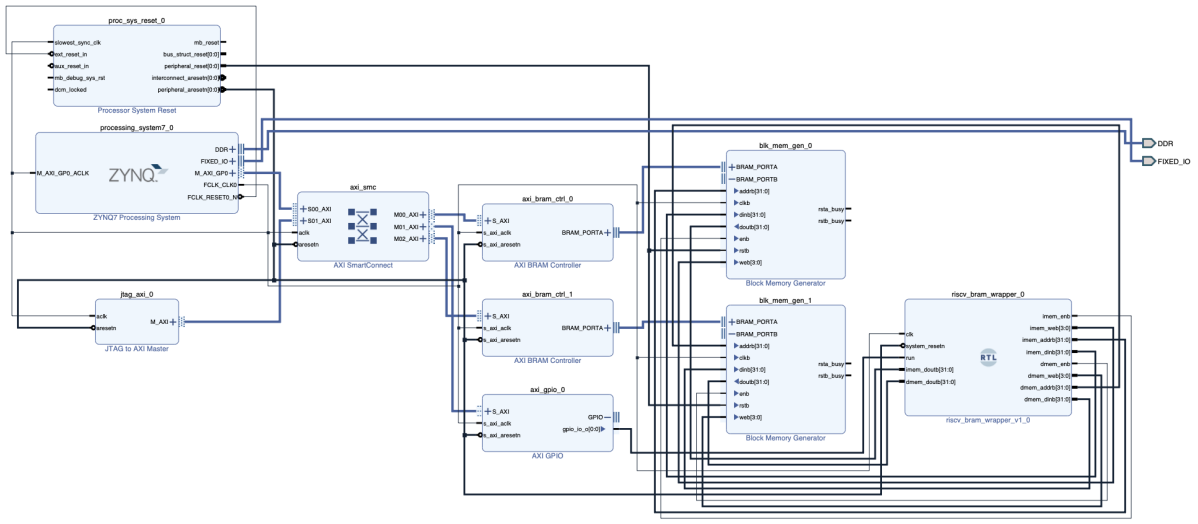

Figure 2 Vivado block design for the PS–PL hardware-in-the-loop system. The PS communicates through AXI SmartConnect, BRAM controllers, and reset GPIO with the BRAM-backed RISC-V wrapper.

Memory and write-back (MEM/WB). The memory stage presents load/store address, write data, and write enable to the data-memory interface. The write-back mux selects either the ALU result or load result and commits it to the selected destination register.

2.2 Hazard control

The hazard unit covers two key pipeline hazards. First, it forwards an arithmetic result from MEM or WB to EX when an instruction needs a value that has not yet reached the register file. Second, a load-use dependency is detected because the loaded value becomes available later than an EX result. In this case, the fetch and decode stages stall for one cycle and a bubble is inserted into EX. Branches and jumps flush younger pipeline state when taken. This separation of forwarding, stalling, and flushing is central to preserving sequential program semantics in a pipelined processor.

2.3 Reset and completion signaling

Reset is a board-level interface rather than only an RTL initialization event. The host drives the AXI GPIO low while BRAM contents are changed and high only after program and data readback pass. This avoids starting the processor from incomplete or stale memory contents. Completion is equally explicit: the notebook accepts a run only if the expected DONE word is seen before the timeout and the full signed output vector equals the reference vector.

3 Testing and Debugging

3.1 Layered verification methodology

The testing sequence was designed to isolate faults close to their origin:

1. Verify that the bitstream, hardware handoff file, and HEX program exist; load the overlay (Figure 1).
2. Check the expected MMIO base addresses against overlay metadata (Figure 3).
3. Test each BRAM independently by write/readback and restore its original value (Figure 4).
4. Inject the program image into instruction BRAM and read it back word for word (Figure 5).
5. Hold reset, write the input vector, clear the DONE word, then release reset (Figure 10).
6. Poll for `0xCAFEBABE` and compare all output words against Python's sorted reference sequence (Figure 11, Figure 12).

3.2 RTL waveform verification

In addition to the board-level checks, RTL simulation was used to inspect the events that are difficult to infer from the final sorted array alone. The traces shown in Figure 6, Figure 7, Figure 8, and Figure 9 were captured from the processor/BRAM interface. They show the program counter, pipeline-valid bits,

PYNQ-Z2 · RISC-V HARDWARE-IN-THE-LOOP VERIFICATION

AXI Address Map and MMIO Initialization

Instruction BRAM, Reset GPIO, and Data BRAM are mapped and accessible.

In [1] · source line 2993

```
EXPECTED_ADDRESSES = {
    0x40000000: "Instruction BRAM",
    0x41200000: "Reset GPIO",
    0x42000000: "Data BRAM",
}

detected_by_address = {
    region["base"]: region
    for region in address_regions
}

print("Address metadata verification:\n")

for expected_address, purpose in EXPECTED_ADDRESSES.items():
    region = detected_by_address.get(expected_address)

    if region is not None:
        print(
            f"PASS: {purpose:20s} "
            f"0x{expected_address:08X} "
            f"({region['source']} → {region['name']})"
        )
    else:
        print(
            f"NOT LISTED: {purpose:20s} "
            f"0x{expected_address:08X}"
        )
```

OUTPUT

```
Address metadata verification:

PASS: Instruction BRAM     0x40000000 (mem_dict → axi_bram_ctrl_0)
PASS: Reset GPIO           0x41200000 (ip_dict → axi_gpio_0)
PASS: Data BRAM            0x42000000 (mem_dict → axi_bram_ctrl_1)
```

In [2] · source line 3031

```
from pynq import MMIO
import time

INSTR_BASE = 0x40000000
RESET_BASE = 0x41200000
DATA_BASE  = 0x42000000

INSTR_RANGE = 0x1000
RESET_RANGE = 0x1000

# Must include data offset 0x0000 and DONE offset 0x1000
DATA_RANGE = 0x2000

instr_mmio = MMIO(INSTR_BASE, INSTR_RANGE)
reset_mmio = MMIO(RESET_BASE, RESET_RANGE)
data_mmio  = MMIO(DATA_BASE, DATA_RANGE)

print("All MMIO objects were created.")
```

OUTPUT

```
All MMIO objects were created.
```

In [3] · source line 3056

```
CPU_STOP = 0x00000000

reset_mmio.write(0x00, CPU_STOP)
time.sleep(0.01)

print("CPU is held in reset.")
```

OUTPUT

```
CPU is held in reset.
```

Figure 3 AXI address-map and MMIO initialization check. The PYNQ overlay reports the expected instruction-BRAM base `0x40000000`, reset-GPIO base `0x41200000`, and data-BRAM base `0x42000000`; the notebook then creates the MMIO interfaces and holds the CPU in reset.

PYNQ-Z2 · RISC-V HARDWARE-IN-THE-LOOP VERIFICATION

BRAM Read/Write Verification

Both Data BRAM and Instruction BRAM pass reversible MMIO tests.

In [1] · source line 3069

```
TEST_PATTERN = 0xA5A55A5A
TEST_OFFSET = 0x0000

original_data_word = int(
    data_mmio.read(TEST_OFFSET)
) & 0xFFFFFFFF

print(f"Original Data BRAM word: 0x{original_data_word:08X}")

try:
    data_mmio.write(TEST_OFFSET, TEST_PATTERN)

    data_readback = int(
        data_mmio.read(TEST_OFFSET)
    ) & 0xFFFFFFFF

    print(f"Written pattern:          0x{TEST_PATTERN:08X}")
    print(f"Read-back value:          0x{data_readback:08X}")

    if data_readback != TEST_PATTERN:
        raise RuntimeError(
            "Data BRAM MMIO test failed."
        )

    print("PASS: Data BRAM is accessible at 0x42000000.")

finally:
    data_mmio.write(TEST_OFFSET, original_data_word)

    restored_value = int(
        data_mmio.read(TEST_OFFSET)
    ) & 0xFFFFFFFF

    print(f"Restored original word:  0x{restored_value:08X}")
```

OUTPUT

```
Original Data BRAM word: 0x00000000
Written pattern:         0xA5A55A5A
Read-back value:         0xA5A55A5A
PASS: Data BRAM is accessible at 0x42000000.
Restored original word:  0x00000000
```

In [2] · source line 3114

```
TEST_PATTERN = 0x12345678
TEST_OFFSET = 0x0000

original_instruction = int(
    instr_mmio.read(TEST_OFFSET)
) & 0xFFFFFFFF

print(
    f"Original Instruction BRAM word: "
    f"0x{original_instruction:08X}"
)

try:
    instr_mmio.write(TEST_OFFSET, TEST_PATTERN)

    instruction_readback = int(
        instr_mmio.read(TEST_OFFSET)
    ) & 0xFFFFFFFF

    print(f"Written pattern:               0x{TEST_PATTERN:08X}")
    print(f"Read-back value:               0x{instruction_readback:08X}")

    if instruction_readback != TEST_PATTERN:
        raise RuntimeError(
            "Instruction BRAM MMIO test failed."
        )

    print(
        "PASS: Instruction BRAM is accessible "
        "at 0x40000000."
    )

finally:
    instr_mmio.write(
        TEST_OFFSET,
        original_instruction
    )

    restored_instruction = int(
        instr_mmio.read(TEST_OFFSET)
    ) & 0xFFFFFFFF

    print(
        f"Restored original instruction:  "
        f"0x{restored_instruction:08X}"
    )
```

OUTPUT

```
Original Instruction BRAM word: 0x00000000
Written pattern:                0x12345678
Read-back value:                0x12345678
PASS: Instruction BRAM is accessible at 0x40000000.
Restored original instruction:  0x00000000
```


Figure 4 Independent AXI BRAM write/readback checks before core execution.

forwarding selects, memory controls, and externally visible BRAM transactions; therefore, they provide direct evidence for the reset, hazard, store, and completion mechanisms described in Section 2.

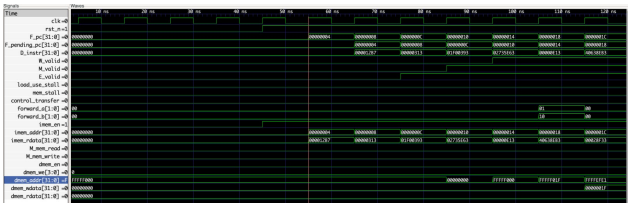

Figure 6 Reset release and pipeline startup (0–120 ns). While `rst_n` is low, the PC, valid bits, and memory controls remain inactive. After reset is deasserted, instruction fetch begins at `0x00000000`; `F_pc` advances in four-byte steps while `F_pending_pc` tracks the synchronous instruction-BRAM request. The successive `E_valid`, `M_valid`, and `W_valid` assertions show the first instructions entering the five-stage pipeline.

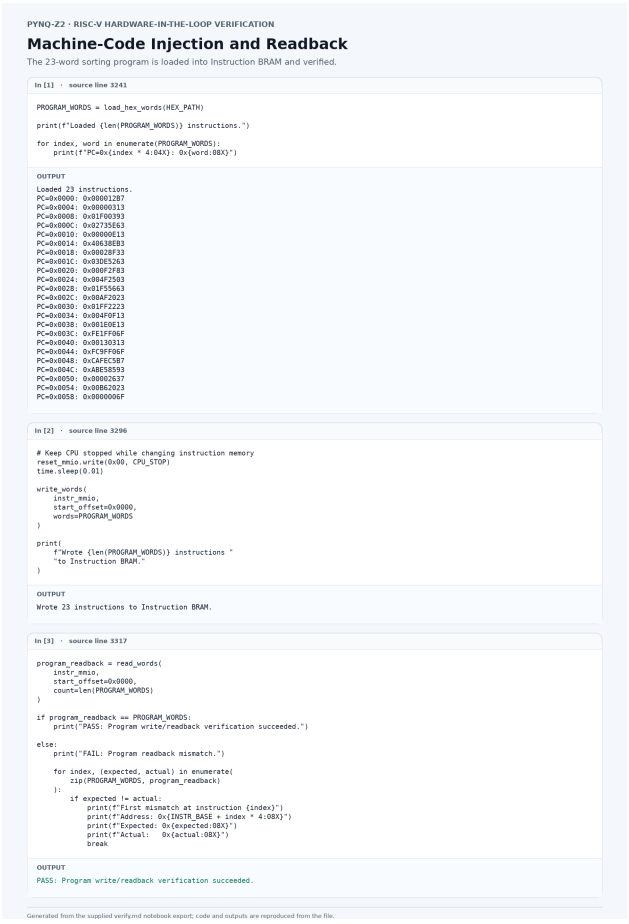

Figure 5 Program injection and instruction-memory readback verification.

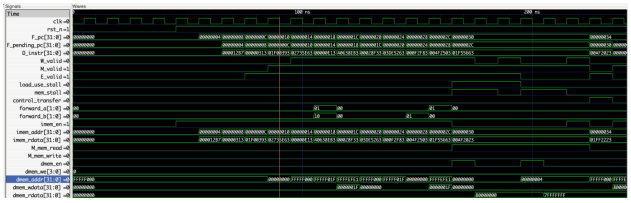

Figure 7 Load-use interlock and synchronous data-BRAM transactions. A load-use dependency holds Fetch/Decode while the required data is returned; the PC does not advance during the asserted stall. The trace also shows the read at local offset `0x00000000` followed by the read at `0x00000004`, with the pipeline released only after the response is available.

3.3 End-to-end board tests

The first full-system test uses reverse-order input as shown in Figure 11, which is demanding for insertion sort because it maximizes the number of comparisons and shifts. The second uses signed values as documented in Figure 12, with negative values, zero, duplicates, and positive values. The latter confirms that the result is interpreted as signed 32-bit data rather than an unsigned ordering. Both notebook captures report a successful comparison (Figure 10, Figure 11, Figure 12).

3.4 Debugging experience and limitations

The most sensitive boundary was the PS–PL contract: a wrong base address, stale DONE word, or reset re-

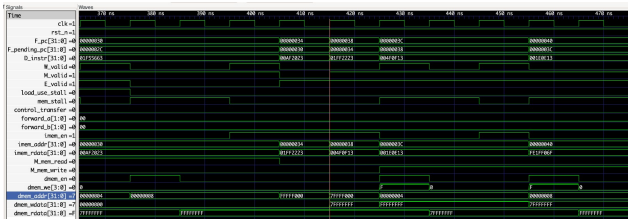

Figure 8 Adjacent-element exchange in the signed sorting loop. Two full-word Port-B write transactions (`dmem_we = 4'hF`) write `0xFFFFFFFF` and `0x7FFFFFFF` to consecutive local data-BRAM offsets. The two pulses demonstrate that each store is issued once, even though the Memory stage remains occupied by back-to-back stores.

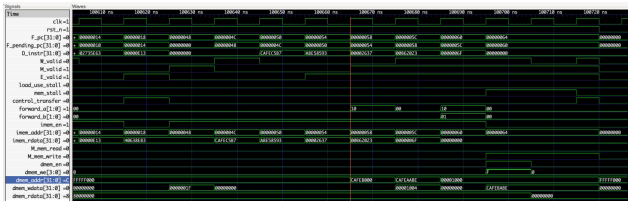

Figure 9 Completion sequence and terminal loop. The instructions `LUI`, `ADDI`, `LUI`, and `SW` construct `0xCAFEBABE` and issue one full-word write at local data-BRAM offset `0x00001000`, corresponding to the reserved DONE location. The following `JAL x0,0` remains in the terminal loop and produces no further data-memory writes.

lease before memory initialization can make a correct core appear faulty. The staged notebook checks directly target these failure modes. RTL simulation validates logic behavior but cannot by itself prove that the generated hardware handoff, AXI address map, reset polarity, BRAM connectivity, routing, and timing constraints agree on the board.

The routed Vivado methodology report also lists five warnings for LUT-driven asynchronous-reset alerts and ten warnings for suboptimally placed synchronized register chains. These are not ignored: they are implementation-quality items that should be reviewed in a production design. However, the routed timing report states that all specified timing constraints are met, and neither warning class prevented the documented on-board tests from passing.

4 FPGA Implementation Analysis

Table 2 reports the placed/routed utilization and timing values for the xc7z020clg400-1 target. The figures include the full integrated design: PS interface, AXI interconnect, BRAM controllers, debug infrastructure, and custom PL logic, not the processor core in isolation.

Figure 10 The host starts the processor, detects the completion word, and stops the core.

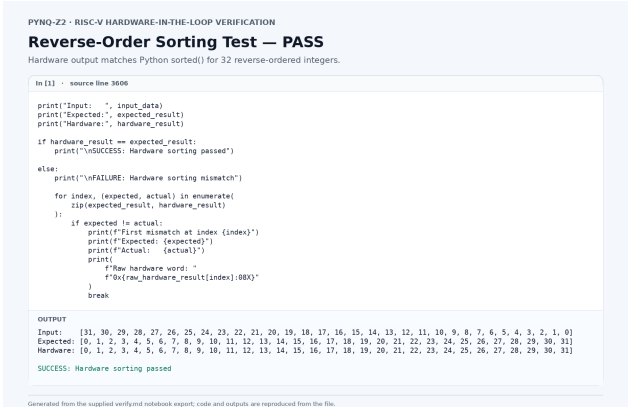

Figure 11 Successful reverse-order sorting test.

Table 2 Routed Vivado implementation results.

Metric	Routed result	Interpretation
Target clock	50.000 MHz (20.000 ns)	PS-generated PL clock constraint
WNS / TNS	+7.081 ns / 0.000 ns	No setup endpoints fail timing
WHS / THS	+0.019 ns / 0.000 ns	No hold endpoints fail timing
Slice LUTs	10,189 / 53,200 (19.15%)	Full integrated design
Slice registers	11,626 / 106,400 (10.93%)	Pipeline, interconnect, and peripheral state
Block RAM tiles	6.5 / 140 (4.64%)	Six RAMB36E1 and one RAMB18E1
DSPs	0 / 220 (0.00%)	No DSP blocks required

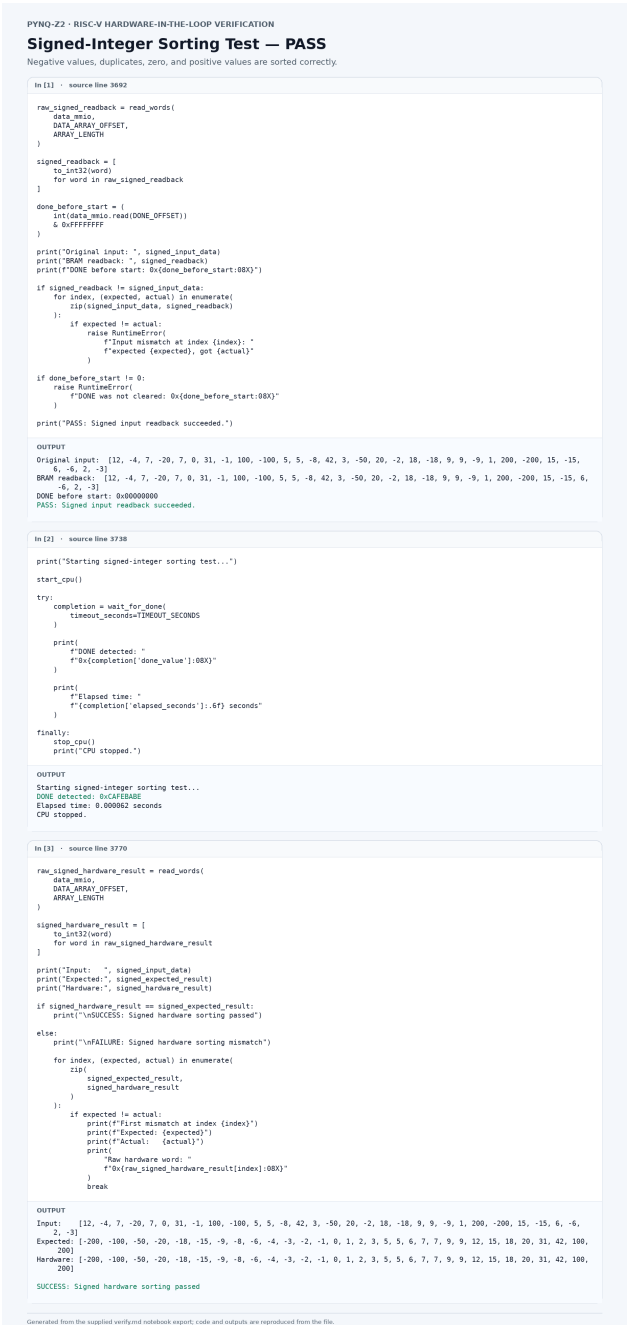

Figure 12 Successful signed-integer sorting test, including negatives and duplicates.

4.1 Resource utilization

Resource figures were collected from the final `impl_1` implementation run using *Open Implemented Design → Reports → Report Utilization → Hierarchy*. This post-placement report is the appropriate source for complete-system resource consumption. In contrast, reports generated for individually synthesized IP blocks describe only the corresponding out-of-context module and must not be presented as complete-system utilization.

The complete-system values in Table 2 were obtained from the post-placement `impl_1` utilization report. They are the appropriate figures for describing the deployed PYNQ-Z2 system because they include the processing-system interface, AXI interconnect, BRAM controllers, debug infrastructure, and processor. The resulting 10,189 LUTs and 11,626 registers correspond to 19.15% and 10.93%, respectively, of the target device. The design uses six RAMB36 blocks and one RAMB18 block (6.5 BRAM tiles) and no DSP slices.

Table 3 records the separate out-of-context synthesis result for the RISC-V wrapper. It is useful for indicating the scale of the custom processor logic, but it is not directly comparable with the placed complete-design result because the two reports are produced at different implementation stages. A direct core-area comparison for the bonus experiment must use post-route utilization reports for both wrappers under identical settings.

For a more detailed processor analysis, the Utilization hierarchy can be expanded along `axi_bram_bd_i` → `riscv_bram_wrapper_0` → `CORE`. Vivado may retain separate entries for decode, execution, arithmetic, register-file, or hazard-management logic. Only hierarchy levels explicitly retained and reported by Vivado should be tabulated; synthesis can flatten small control and forwarding functions, in which case a separate value cannot be determined reliably and is not estimated.

Table 3 RISC-V wrapper utilization from out-of-context synthesis.

Resource	Utilized
Slice LUTs	1,668
LUTRAMs	0
Slice registers	1,594
RAMB36	0
RAMB18	0

4.2 Timing interpretation

At the constrained 20.000 ns period, the +7.081 ns worst setup slack means the slowest constrained setup path retains 7.081 ns of margin. The correct conclusion is that this implementation closes timing at 50 MHz. It would be incorrect to compute a measured maximum clock rate simply as $1/(20\text{ ns} - \text{WNS})$, because the reported slack belongs to this particular constraint and implementation. A real $F_{\max}$ measurement requires rerunning implementation with tighter clock periods until timing closure is lost.

4.2.1 Maximum Operating Frequency

Maximum Operating Frequency:

$$F_{\max} = \frac{1}{T_{\text{clock}} - \text{WNS}} \tag{1}$$

4.3 Optional Bonus: Pipelined versus single-cycle core

4.3.1 Controlled comparison methodology

A valid comparison changes only the microarchitecture. Two cores must implement the same documented instruction subset, start from the same reset vector, execute the same program image, and use the same instruction/data BRAM configuration and memory latency. They must be synthesized and implemented for the same xc7z020clg400-1 device with the same Vivado version, clock definitions, XDC constraints, optimization directives, and debug-core policy. The same input vectors — reverse ordered, signed/duplicate, randomized, and boundary-value arrays — must be used. Both cores must pass the complete functional suite before any performance result is reported.

The implemented board uses the two-individual-BRAM organization shown in Figure 2: instruction BRAM has a PS access port and a core fetch port, while true-dual-port data BRAM has a PS access port and a core read/write port. Consequently, the current core may fetch an instruction while its data-memory stage accesses D-RAM; it does *not* require a fetch-versus-load/store arbiter. This point must remain fixed for both cores in the main comparison. A separate experiment may compare the unified-DP-BRAM option, but then both designs must use the same one-core-port constraint and count the arbitration/stall cycles as part of the measured result.

The processor start and stop points must also be identical. A hardware cycle counter should be cleared before reset release and read after the DONE word is written; host polling time must *not* be used as execution time because it includes Linux scheduling, AXI transactions, and the polling interval. At least 30 repetitions per vector should be recorded when board measurements are used, with the median reported. The reported design point for each core is the fastest clock period that achieves non-negative setup and hold slack after routing, rather than an arbitrary common frequency.

A concise result table should list, for each benchmark, $T_{\min}$, $F_{\max}$, C, t, IPC, and post-route LUT/FF/BRAM totals for both cores. The speedup is then

$$S = \frac{t_{\text{single}}}{t_{\text{pipe}}} = \frac{C_{\text{single}}/F_{\text{single}}}{C_{\text{pipe}}/F_{\text{pipe}}}.$$

This formulation prevents a misleading result in which a pipeline has fewer cycles but a lower clock rate, or vice versa.

4.3.2 Architecture trade-off

A single-cycle core performs fetch, decode, register read, ALU operation, data-memory access, and write-back in one clock period. Its critical path therefore contains the sum of these operations and the associated multiplexers. It has no inter-stage registers, forwarding network, load-use interlock, or branch-flush control. Consequently, it usually uses fewer flip-flops and may have simpler control wiring. It may also use fewer LUTs, but this is not guaranteed: a long one-cycle data path can require wider or more complex selection logic after synthesis.

The pipelined core places registers between these functions. Its clock period is limited by the slowest stage plus register overhead, rather than by the entire instruction path; this normally reduces logic depth and permits a higher $F_{\max}$. The price is visible area and control overhead: pipeline registers increase FF count, forwarding multiplexers add logic, and hazard/flush control adds comparators and enables. A load-use dependency introduces at least one stall in the present

Table 4 Metrics for a fair pipelined-versus-single-cycle comparison. Values for the single-cycle reference must come from its own routed implementation and hardware counter.

Metric	Measurement	Why it matters
Minimum closing period $T_{\min}$ and $F_{\max} = 1/T_{\min}$	Clock-period sweep; use routed WNS/WHS	Measures the effect of logic depth
Benchmark cycles C	Counter between reset release and DONE write	Separates architectural stalls from clock rate
Execution time $t = C/F$	Derived from the hardware cycle count and achieved frequency	Primary performance metric
Retired instructions and IPC	IPC = N_{retired}/C	Shows branch and load-use penalties
LUTs, FFs, BRAMs, DSPs	Routed utilization report	Quantifies implementation cost
Critical path	Timing-path report	Identifies the limiting logic cone
Energy, if available	Board power times execution time	Extends comparison beyond speed and area

five-stage organization, and a resolved taken branch or jump discards younger instructions. Thus its ideal CPI of one is not achieved by all programs.

The insertion-sort workload illustrates this trade-off especially well. Its inner loop repeatedly loads a value, compares it, conditionally branches, and stores a shifted value. The load-to-use dependency and frequent conditional branches reduce pipeline IPC, so a five-stage pipeline should not be claimed to be five times faster. Nevertheless, if the higher clock frequency outweighs the additional stalls and branch penalties, the pipelined core has lower total execution time. If the benchmark is short or branch-heavy and the clock-rate advantage is small, a simpler single-cycle core can be competitive despite its longer logic path.

4.3.3 Implemented bonus infrastructure and current evidence

The bonus RTL package adds a single-cycle RV32I reference core and a drop-in BRAM wrapper alongside the pipelined wrapper. Both use the same byte-addressed instruction/data BRAM Port-B interfaces, reset/run protocol, program image, and DONE-word convention. The pipelined implementation also contains cycle, retired-instruction, load, and store counters; the single-cycle reference provides cycle and instruction-retirement counters and supports `rdcycle` and `rdinstret` for software-visible measurements. This establishes the infrastructure needed to measure CPI rather than relying on host polling time.

For the implemented pipelined system, the routed design closes at 50 MHz with +7.081 ns WNS and the end-to-end sorting tests pass. A matched routed single-cycle bitstream and corresponding post-route utilization/timing reports have not yet been generated. Consequently, this report presents the verified pipelined evidence and the controlled comparison method, but does not invent single-cycle resource, frequency, cycle, or speedup values. Once both wrappers are implemented with the same constraint sweep, the resulting measurements can be entered in Table 4.

5 Potential Improvements

5.1 Core and microarchitecture

The first improvement would be to complete and document a broader RV32I instruction subset, including all branch variants, and to expose the added hardware counters through a stable software-visible interface in both implementations. This would permit cycle-accurate benchmarking instead of using host polling time, which includes software and AXI overhead. The next measurement step is to route the matched single-cycle core with the same memories, benchmark, constraints, and FPGA target. Comparing achieved clock period, cycles, execution time, and LUT/FF/BRAM usage would then give a defensible quantitative pipeline-versus-single-cycle result.

Control-flow performance could be improved by resolving simple branches earlier in the pipeline or by adding a small static or dynamic branch predictor. The current design correctly flushes younger instructions after a taken control transfer, but every resolved branch creates a penalty. An instruction prefetch buffer and small instruction/data caches are natural next steps once the basic pipeline is stable. For larger programs, small direct-mapped instruction and data caches would reduce BRAM or external-memory access pressure, although their miss handling and coherence assumptions would add significant verification complexity.

5.2 Timing, reset, and physical implementation

The reported positive slack shows that the present 50 MHz target is safe, but it does not establish the maximum achievable frequency. A scripted clock-period sweep should be used to tighten the constraint until setup timing fails; this would characterize the actual timing limit and identify the dominant path. The asynchronous-reset and synchronizer-placement warnings should also be addressed by reviewing reset fan-out, using synchronized reset deassertion in the relevant clock domain, and applying placement guidance only where it is technically justified. These changes would improve implementation robustness even though the current board tests pass.

5.3 Verification and observability

The notebook already performs strong end-to-end tests; it can be extended with randomized signed arrays, boundary values such as `0x80000000` and `0x7fffffff`, repeated runs after reset, and deliberately injected invalid program images. At RTL level, self-checking testbenches, SystemVerilog assertions for stall/flush behavior, and coverage of forwarding and load-use cases would make regressions easier to locate. Finally, hardware performance counters, an instruction-retire trace, or an integrated logic analyzer would make on-board failures diagnosable without relying only on the final DONE word and memory result.

6 Conclusion

The project meets its core objective: a pipelined RISC-V processor is integrated into the PYNQ-Z2 PS–PL system and verified through an automated on-board workflow. The test process verifies overlay loading, AXI addressability, independent BRAM access, program injection, reset sequencing, completion signaling, and signed result checking. The routed design meets the 50 MHz constraint, and both supplied end-to-end sorting tests pass.

The most important HDL lessons are that pipeline correctness needs explicit handling of data and control dependencies, and system correctness needs a precise hardware/software contract. The bonus RTL establishes a matched single-cycle reference and hardware-counter infrastructure; future work should route that reference, run a clock-period sweep to establish $F_{\max}$, and capture assertion-based hazard tests. Extensions such as instruction/data caches, branch prediction, or more comprehensive performance counters would also be natural next steps.

References

References

[1] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, available at https://riscv.org/technical/specifications/.

[2] AMD Xilinx, *Vivado Design Suite User Guide: Design Analysis and Closure Techniques (UG906)*, 2021.